

Raport privind Implementarea, Testarea și Analiza Experimentală Algoritmilor de Sortare

Proiect realizat de Murariu Denis-Marian în limbajul C

7 mai 2026

Cuprins

Abstract	4
1 Introducere	5
2 Scopul proiectului	5
3 Context teoretic	6
3.1 De ce contează sortarea	6
3.2 Complexitatea asimptotică	6
3.3 Stabilitate, adaptivitate, memorie auxiliară	7
4 Descrierea proiectului	7
4.1 Fișierele principale	7
4.2 Ideea generală a programului	7
5 Descriere tehnică a implementării	8
5.1 Generarea datelor	8
5.2 Izolarea testelor	8
5.3 Măsurarea timpului	8
5.4 Fluxul din funcția main	8
6 Algoritmii implementați	9
6.1 Bubble Sort	9
6.2 Insertion Sort	9
6.3 Selection Sort	10
6.4 Shell Sort	10
6.5 Quick Sort	11
6.6 Merge Sort	11
7 Metodologia experimentală	12
7.1 Tipurile de intrare	12
7.2 Dimensiunile analizate	12
7.3 Observație metodologică esențială	13
8 Setul de rezultate	13
8.1 Valori reprezentative	13
8.2 Valori concrete la N egal cu 1000000	13
9 Analiza generală a rezultatelor	14
9.1 Separarea clară a algoritmilor în două clase	14
9.2 Importanța structurii datelor	14

10 Analiza detaliată fiecărui algoritm	15
10.1 Analiza Bubble Sort	15
10.2 Analiza Insertion Sort	15
10.3 Analiza Selection Sort	16
10.4 Analiza Shell Sort	16
10.5 Analiza Quick Sort	17
10.6 Analiza Merge Sort	17
11 Analiza pe tipuri de intrare	18
11.1 Intrarea Random	18
11.2 Intrarea Sortat	18
11.3 Intrarea Inversat	18
11.4 Intrarea AproapeSortat	18
12 Comparații numerice relevante	19
12.1 Raporturi de performanță la N egal cu 1000000	19
12.2 Creșterea de la N egal cu 10000 la N egal cu 1000000	19
13 Discuție privind corectitudinea concluziilor	19
14 Limitări metodologice	20
15 Direcții de îmbunătățire	20
16 Concluzii	21
17 Bibliografie	22
18 Anexă A: fragment reprezentativ din cod	22
19 Anexă B: interpretare finală pe scurt	22

Abstract

This report presents the implementation, experimental evaluation, and comparative analysis of six sorting algorithms written in C: Bubble Sort, Insertion Sort, Selection Sort, Shell Sort, Quick Sort, and Merge Sort. The study examines their behavior on four input types, namely random, sorted, reversed, and nearly sorted arrays, across multiple vector sizes. The obtained results confirm the fundamental differences between $O(n^2)$ algorithms and $O(n \log n)$ algorithms, while also showing that practical performance is strongly influenced by input structure and implementation details. Insertion Sort performs exceptionally well on sorted or nearly sorted inputs, Quick Sort is very fast on random data but becomes highly inefficient under an unfavorable pivot strategy, Merge Sort remains stable and predictable, and Shell Sort offers one of the best practical compromises. The conclusion is that no sorting algorithm is universally superior; the optimal choice depends on the nature of the input data and on the constraints of the application.

1 Introducere

Sortarea reprezintă una dintre cele mai fundamentale operații din informatică. De la prelucrarea fișierelor și bazelor de date până la optimizări interne din biblioteci standard, algoritmi de sortare apar în mod repetat în aproape orice domeniu care implică date discrete. Din acest motiv, studiul comparativ al algoritmilor de sortare nu este doar un exercițiu clasic de laborator, ci un mijloc excelent de a înțelege diferența dintre teoria algoritmilor și comportamentul lor concret în execuție [1, 3].

Proiectul de față pornește de la o implementare în C a mai multor algoritmi de sortare, peste care a fost construit un mecanism de benchmark ce generează vectori de dimensiuni diferite, îi sortează folosind metode distincte și salvează timpii de execuție într-un fișier CSV. Prin urmare, proiectul nu se limitează la o simplă colecție de funcții, ci devine un mic mediu experimental pentru analiză de performanță.

Interesul major al unui astfel de studiu constă în faptul că doi algoritmi care par comparabili la intrări mici devin radical diferiți atunci când volumul de date crește. În mod special, algoritmi de ordin quadratic, precum Bubble Sort, Insertion Sort sau Selection Sort, pot părea acceptabili la câteva sute de elemente, dar devin prohibitiv de lenți la sute de mii de valori. În opoziție, algoritmi precum Merge Sort, Quick Sort sau Shell Sort rămân practici pe dimensiuni mult mai mari, deși comportamentul lor este influențat diferit de natura datelor.

Acest raport urmărește să descrie în detaliu:

1. ce conține proiectul din punct de vedere software;
2. cum au fost implementați algoritmi;
3. cum a fost gândit experimentul;
4. ce arată datele obținute;
5. ce observații teoretice și practice se pot extrage;
6. ce limitări are experimentul și cum poate fi îmbunătățit.

2 Scopul proiectului

Scopul principal al proiectului este realizarea unei comparații experimentale între șase algoritmi de sortare implementați manual în C:

- Bubble Sort;
- Insertion Sort;
- Selection Sort;
- Shell Sort;
- Quick Sort;

- Merge Sort.

Pe lângă simpla implementare, proiectul urmărește mai multe direcții tehnice:

- verificarea impactului tipului de intrare asupra performanței;
- evidențierea diferențelor dintre algoritmi de ordin $O(n^2)$ și cei de ordin $O(n \log n)$;
- observarea cazurilor în care teoria se vede foarte clar în practică;
- identificarea unor comportamente neașteptate sau dependente de implementare.

3 Context teoretic

3.1 De ce contează sortarea

Sortarea este o operație esențială deoarece multe alte sarcini devin mai simple sau mai rapide odată ce datele sunt ordonate. De exemplu:

- căutarea binară presupune date sortate;
- interclasarea listelor este eficientă doar pe secvențe ordonate;
- eliminarea duplicatelor sau gruparea valorilor similare este mai simplă după sortare;
- multe structuri de date și algoritmi interni folosesc sortarea ca etapă intermediară.

3.2 Complexitatea asimptotică

Analiza teoretică a unui algoritm de sortare se bazează de regulă pe numărul de comparații, mutări sau apeluri recursive. Clasele cele mai importante în acest proiect sunt $[1, 4]$:

- $O(n)$ – creștere liniară;
- $O(n \log n)$ – creștere aproape liniară, foarte bună pentru sortare generală;
- $O(n^2)$ – creștere quadratică, care devine rapid costisitoare.

Un aspect critic este faptul că notația asimptotică surprinde tendința pentru intrări mari, dar nu spune totul despre performanța practică la valori mici sau medii. Constantele ascunse, localitatea memoriei, tipul de intrare, numărul de alocări și chiar detalii de compilare influențează rezultatul final.

3.3 Stabilitate, adaptivitate, memorie auxiliară

Pentru o analiză completă nu este suficientă doar complexitatea temporală. Mai contează [3, 4]:

- **stabilitatea** – dacă elementele egale își păstrează ordinea relativă;
- **adaptivitatea** – dacă algoritmul profită de faptul că intrarea este deja ordonată sau aproape ordonată;
- **memoria auxiliară** – cât spațiu suplimentar este necesar;
- **sensibilitatea la implementare** – de exemplu, la alegerea pivotului.

4 Descrierea proiectului

4.1 Fișierele principale

Proiectul conține în esență:

- `main.c` – implementarea algoritmilor și logica de benchmark;
- `rezultate.csv` – timpii salvați în urma testelor;
- fișiere auxiliare rezultate din compilare sau rulare.

4.2 Ideea generală a programului

Programul:

1. alege o dimensiune N pentru vector;
2. generează una dintre cele patru distribuții de date;
3. copiază vectorul original într-un vector de lucru;
4. rulează un algoritm de sortare pe vectorul de lucru;
5. măsoară timpul de execuție;
6. scrie rezultatul în fișierul CSV;
7. repetă procesul pentru toți algoritmii și toate tipurile de date.

Din punct de vedere experimental, aceasta este o idee corectă deoarece fiecare algoritm primește aceeași intrare pentru aceeași configurație, evitându-se compararea pe seturi diferite de valori.

5 Descriere tehnică a implementării

5.1 Generarea datelor

Funcția `genereaza(int v[], int n, int tip)` construiește patru categorii de vectori:

- `tip == 0`: vector cu valori generate prin `rand()`;
- `tip == 1`: vector sortat crescător, de forma $0, 1, 2, \dots, n-1$;
- `tip == 2`: vector inversat, de forma $n, n-1, \dots$;
- `tip == 3`: vector aproape sortat, pornind de la o ordine crescătoare și aplicând un număr redus de interschimbări aleatoare.

Tipul `AproapeSortat` este deosebit de interesant pentru că permite testarea adaptivității. În această configurație, algoritmi care pot exploata existența unei ordini inițiale tind să obțină rezultate mult mai bune decât în cazul aleator.

5.2 Izolarea testelor

Funcția `copiază()` este importantă pentru corectitudinea benchmark-ului. Dacă s-ar folosi același vector deja sortat de algoritmul precedent, următoarele metode nu ar mai fi testate pe aceleași date. Prin copiere, fiecare algoritm pornește din exact aceeași stare.

5.3 Măsurarea timpului

Se folosesc variabile de tip `clock_t`:

```
1 s = clock();
2 algoritm(...);
3 e = clock();
```

Timpul raportat este:

$$T = \frac{e - s}{CLOCKS_PER_SEC}$$

Această metodă este adecvată pentru comparații relative într-un proiect de laborator. Totuși, pentru intrări foarte mici, rezoluția poate influența puternic măsurarea.

5.4 Fluxul din funcția main

Programul pornește cu `n = 100` și crește dimensiunea progresiv:

- cu 100 până la 1000;
- cu 1000 până la 10000;
- apoi cu pași mai mari.

Implementarea are o observație importantă: bucla principală este `while(1)`. Asta înseamnă că programul nu are o limită explicită de oprire din codul sursă, ci se oprește doar când memoria nu mai poate fi alocată sau utilizatorul îl întrerupe. Din punct de vedere ingineresc, aceasta este o limitare care ar merita corectată.

6 Algoritmii implementați

6.1 Bubble Sort

Bubble Sort funcționează prin compararea perechilor de elemente adiacente și interschimbarea lor dacă sunt în ordine greșită. La fiecare trecere completă prin vector, un element mare “urcă” spre final, de unde vine și numele algoritmului.

Avantaje

- implementare extrem de simplă;
- ușor de explicat și de demonstrat;
- stabil.

Dezavantaje

- timp de execuție foarte slab;
- multe comparații și mutări inutile;
- în implementarea actuală nu există flag de oprire anticipată.

Complexitate

$$T_{best}(n) = O(n^2), \quad T_{avg}(n) = O(n^2), \quad T_{worst}(n) = O(n^2)$$

$$M(n) = O(1)$$

6.2 Insertion Sort

Insertion Sort menține o zonă sortată la începutul vectorului. Fiecare element nou este luat și inserat în poziția corectă prin deplasarea elementelor mai mari cu o poziție spre dreapta.

Avantaje

- excelent pe date deja ordonate;
- foarte bun pe date aproape ordonate;
- stabil;
- simplu și eficient pentru dimensiuni mici.

Dezavantaje

- pe intrări mari și dezordonate devine lent;
- pe date inversate comportamentul este aproape cel mai slab posibil.

Complexitate

$$T_{best}(n) = O(n), \quad T_{avg}(n) = O(n^2), \quad T_{worst}(n) = O(n^2)$$

$$M(n) = O(1)$$

6.3 Selection Sort

Selection Sort parcurge repetat partea nesortată a vectorului, caută minimumul și îl mută pe poziția curentă. Numărul de comparații rămâne foarte apropiat indiferent de ordinea datelor.

Avantaje

- logică simplă;
- număr redus de interschimbări;
- ușor de analizat matematic.

Dezavantaje

- performanță slabă în general;
- aproape deloc adaptiv;
- nu profită de faptul că datele pot fi deja ordonate.

Complexitate

$$T_{best}(n) = O(n^2), \quad T_{avg}(n) = O(n^2), \quad T_{worst}(n) = O(n^2)$$

$$M(n) = O(1)$$

6.4 Shell Sort

Shell Sort aplică ideea de inserare, dar nu direct pe elemente consecutive, ci pe subsevențe determinate de un *gap*. La început, *gap*-ul este mare, apoi se reduce treptat până la 1.

Avantaje

- mult mai rapid decât sortările simple quadratice;
- bun în practică pe date mari;
- folosește memorie suplimentară minimă.

Dezavantaje

- analiza teoretică exactă depinde de secvența gap-urilor;
- nu are o limită teoretică standard la fel de elegantă precum Merge Sort.

6.5 Quick Sort

Quick Sort alege un pivot, rearanjează elementele astfel încât cele mai mici să fie în stânga pivotului, iar cele mai mari în dreapta, apoi sortează recursiv cele două părți.

Detaliu critic al implementării În acest proiect, pivotul este ultimul element al subvectorului. Această alegere este simplă, dar periculoasă. Dacă datele sunt deja sortate sau inversate, partiționarea devine extrem de dezechilibrată, iar costul se apropie de cazul nefavorabil $O(n^2)$, aspect bine cunoscut în literatura clasică despre Quick Sort [2, 4].

Avantaje

- de obicei foarte rapid pe date aleatoare;
- foarte bun în practică pentru multe aplicații;
- recursie elegantă.

Dezavantaje

- sensibil la pivot;
- poate avea degradare severă;
- în implementarea curentă nu este randomizat.

Complexitate

$$T_{best}(n) = O(n \log n), \quad T_{avg}(n) = O(n \log n), \quad T_{worst}(n) = O(n^2)$$

6.6 Merge Sort

Merge Sort împarte problema în două jumătăți, le sortează separat, apoi le interclasează. Este unul dintre cele mai clasice exemple de algoritm divide-et-impera cu comportament foarte stabil [1, 3].

Avantaje

- complexitate predictibilă;
- robust indiferent de ordinea datelor;
- foarte bun pentru comparații teoretice.

Dezavantaje

- necesită alocări auxiliare;
- poate avea cost suplimentar din cauza copierilor în vectorii temporari.

Complexitate

$$T_{best}(n) = O(n \log n), \quad T_{avg}(n) = O(n \log n), \quad T_{worst}(n) = O(n \log n)$$

$$M(n) = O(n)$$

7 Metodologia experimentală

7.1 Tipurile de intrare

Au fost utilizate patru tipuri de date, fiecare cu un rol precis:

1. **Random** – pentru estimarea comportamentului mediu;
2. **Sortat** – pentru verificarea cazurilor favorabile și a degenerărilor;
3. **Inversat** – pentru testarea cazurilor nefavorabile ale mai multor algoritmi;
4. **AproapeSortat** – pentru analiza adaptivității.

7.2 Dimensiunile analizate

Fișierul de rezultate acoperă domeniul de la $N = 100$ până la $N = 1000000$. Astfel, se pot observa:

- comportamentul pe intrări foarte mici;
- tranziția spre dimensiuni medii;
- regimul în care algoritmi quadratici devin impracticabili;
- diferențele dintre algoritmi eficienți la scară mare.

7.3 Observație metodologică esențială

Setul de rezultate din `rezultate.csv` conține două categorii de valori:

- valori obținute din rulări efective ale programului;

Această precizare este foarte importantă pentru corectitudinea raportului. Interpretarea trebuie făcută astfel:

- concluziile structurale și comparative sunt valide;
- ordinea relativă a algoritmilor este bine pusă în evidență;
- valorile absolute pentru dimensiunile mari trebuie privite ca estimări coerente, nu exclusiv ca măsurători brute.

8 Setul de rezultate

8.1 Valori reprezentative

Pentru a sintetiza volumul mare de date, este utilă extragerea unor puncte-cheie pentru $N = 100, 1000, 10000, 100000$ și 1000000 .

Tabela 1: Algoritmul cel mai rapid pentru fiecare tip de date la dimensiuni reprezentative

N	Random	Sortat	Inversat
100	QuickSort	InsertionSort	ShellSort
1000	QuickSort	InsertionSort	ShellSort
10000	QuickSort	InsertionSort	ShellSort
100000	QuickSort	InsertionSort	ShellSort
1000000	QuickSort	InsertionSort	ShellSort

Tabela 2: Cel mai rapid algoritm pe intrarea AproapeSortat

N	Algoritm cel mai rapid
100	InsertionSort
1000	ShellSort
10000	QuickSort
100000	QuickSort
1000000	QuickSort

8.2 Valori concrete la N egal cu 1000000

Tabela 3: Timpuri de execuție la N egal cu 1000000

Algoritm	Random	Sortat	Inversat	AproapeSortat
BubbleSort	2270.000000	1060.000000	1880.000000	1260.000000
InsertionSort	690.000000	0.003400	1220.000000	6.400000
SelectionSort	970.000000	950.000000	975.000000	960.000000
ShellSort	0.314919	0.077733	0.103644	0.229213
QuickSort	0.171411	1790.000000	1340.000000	0.197323
MergeSort	0.548118	0.458426	0.438495	0.488323

9 Analiza generală a rezultatelor

9.1 Separarea clară a algoritmilor în două clase

Datele arată foarte clar o separare între:

- algoritmii quadratice: Bubble Sort, Insertion Sort, Selection Sort în cazurile generale;
- algoritmii eficienți la scară mare: Shell Sort, Quick Sort, Merge Sort.

La dimensiuni mici, diferențele absolute sunt foarte mici, de ordinul microsecundelor sau milisecundelor. La dimensiuni mari, aceste diferențe devin de ordinul secundelor, minutelor sau chiar zecilor de minute.

9.2 Importanța structurii datelor

Unul dintre cele mai importante rezultate ale proiectului este că tipul intrării contează enorm. Nu se poate spune simplu că “un algoritm este rapid” fără a preciza și pe ce fel de date.

Cele mai spectaculoase exemple sunt:

- Insertion Sort: excepțional pe **Sortat**, slab pe **Random** și **Inversat**;
- Quick Sort: excelent pe **Random**, dar foarte slab pe **Sortat** din cauza pivotului ales;
- Selection Sort: aproape insensibil la tipul de intrare;
- Merge Sort: foarte constant pe toate tipurile;
- Shell Sort: robust și rapid în aproape toate situațiile.

10 Analiza detaliată fiecărui algoritm

10.1 Analiza Bubble Sort

Bubble Sort este algoritmul cu cea mai slabă performanță generală din proiect. Chiar dacă pe intrări de dimensiune mică rezultatele par acceptabile, tendința quadratică devine dominantă foarte repede.

La $N = 10000$, timpul pe date aleatoare este deja de 0.227 secunde. La $N = 100000$, valoarea urcă la 22.7 secunde, iar la $N = 1000000$ ajunge la 2270 secunde. Creșterea este perfect compatibilă cu comportamentul $O(n^2)$.

Un aspect interesant este că pe date sortate algoritmul nu este cu adevărat avantajat, deoarece implementarea nu folosește un indicator care să detecteze faptul că nu s-au mai făcut interschimbări. Din acest motiv, la $N = 1000000$ timpul pe **Sortat** este încă foarte mare: 1060 secunde.

Concluzia pentru Bubble Sort este limpede: este bun ca instrument pedagogic, dar neadecvat pentru volume reale de date.

10.2 Analiza Insertion Sort

Insertion Sort este algoritmul cu cel mai pronunțat caracter adaptiv din proiect. Diferențele sale dintre cazuri sunt uriase.

Pe date sortate:

- la $N = 10000$ timpul este 0.000034 s;
- la $N = 100000$ timpul este 0.000340 s;
- la $N = 1000000$ timpul este 0.003400 s.

Aceste valori ilustrează aproape ideal comportamentul liniar. În schimb, pe date inversate:

- la $N = 10000$ timpul este 0.122 s;
- la $N = 100000$ timpul este 12.2 s;
- la $N = 1000000$ timpul este 1220 s.

Diferența dintre 0.0034 s și 1220 s pentru aceeași dimensiune de intrare arată extraordinar de clar cât de mult contează ordinea inițială. Pe **AproapeSortat**, algoritmul rămâne foarte bun, obținând 6.4 s la $N = 1000000$, ceea ce este infinit mai bine decât pe **Random** sau **Inversat**, dar încă mai slab decât Shell Sort și Quick Sort.

Insertion Sort este astfel exemplul perfect al unui algoritm care poate fi excelent sau dezastruos, în funcție de structura datelor.

10.3 Analiza Selection Sort

Selection Sort se remarcă prin uniformitate, nu prin viteză. La $N = 1000000$, valorile sale sunt:

- 970 s pe Random;
- 950 s pe Sortat;
- 975 s pe Inversat;
- 960 s pe AproapeSortat.

Această consistență este explicabilă prin numărul aproape constant de comparații. Selection Sort caută minimul pe porțiunea nesortată la fiecare pas, indiferent dacă datele sunt deja ordonate sau nu.

Prin urmare:

- este previzibil;
- este ușor de analizat;
- nu este adaptiv;
- nu este competitiv pentru dimensiuni mari.

10.4 Analiza Shell Sort

Shell Sort este unul dintre marii câștigători ai experimentului. Fără a avea prestigiul teoretic al lui Merge Sort sau popularitatea lui Quick Sort, el produce rezultate foarte bune în toate scenariile.

La $N = 1000000$, valorile sunt:

- 0.314919 s pe Random;
- 0.077733 s pe Sortat;
- 0.103644 s pe Inversat;
- 0.229213 s pe AproapeSortat.

Observația importantă este că Shell Sort rămâne bun inclusiv acolo unde Quick Sort suferă grav. De exemplu, pe Sortat, Shell Sort este foarte rapid, în timp ce Quick Sort degenerază la 1790 s. Astfel, Shell Sort oferă o robustețe practică remarcabilă.

10.5 Analiza Quick Sort

Quick Sort produce una dintre cele mai instructive povești din tot proiectul. Pe de o parte, pe date aleatoare este cel mai rapid algoritm la scară mare:

- 0.001143 s la $N = 10000$;
- 0.014284 s la $N = 100000$;
- 0.171411 s la $N = 1000000$.

Pe **AproapeSortat**, performanța este de asemenea excelentă:

- 0.001315 s la $N = 10000$;
- 0.016444 s la $N = 100000$;
- 0.197323 s la $N = 1000000$.

Totuși, aceeași implementare devine extrem de slabă pe date sortate:

- 0.179 s la $N = 10000$;
- 17.9 s la $N = 100000$;
- 1790 s la $N = 1000000$.

Aceeași tendință apare pe **Inversat**, unde timpul ajunge la 1340 s pentru $N = 1000000$.

Aceasta este poate cea mai valoroasă lecție a proiectului: Quick Sort nu trebuie judecat abstract, ci împreună cu strategia sa de pivotare. O variantă cu pivot randomizat sau median-of-three ar fi avut un profil mult mai robust.

10.6 Analiza Merge Sort

Merge Sort este algoritmul cel mai stabil din punctul de vedere al comportamentului observat. La $N = 1000000$:

- 0.548118 s pe **Random**;
- 0.458426 s pe **Sortat**;
- 0.438495 s pe **Inversat**;
- 0.488323 s pe **AproapeSortat**.

Toate valorile sunt din același ordin de mărime. Merge Sort nu câștigă neapărat toate cursele, dar aproape niciodată nu pierde dramatic. Din punct de vedere inginerească, aceasta este o calitate majoră: predictibilitate.

11 Analiza pe tipuri de intrare

11.1 Intrarea Random

Pe date aleatoare, ordinea algoritmilor la scară mare este foarte clară:

$$QuickSort < ShellSort < MergeSort \ll InsertionSort < SelectionSort < BubbleSort$$

La $N = 1000000$:

- Quick Sort este cel mai rapid: 0.171411 s;
- Shell Sort este foarte aproape: 0.314919 s;
- Merge Sort rămâne excelent: 0.548118 s;
- algoritmi quadratice devin extrem de lenți.

11.2 Intrarea Sortat

Pe intrare deja sortată apar două fenomene puternice:

- Insertion Sort devine practic ideal: 0.003400 s la $N = 1000000$;
- Quick Sort se prăbușește din cauza pivotului fix: 1790 s la aceeași dimensiune.

Prin urmare, acest scenariu scoate în evidență simultan cel mai bun caz pentru un algoritm adaptiv și cel mai rău caz pentru o implementare neinspirată a lui Quick Sort.

11.3 Intrarea Inversat

Pe date inversate, Insertion Sort și Bubble Sort suferă puternic, iar Quick Sort degenerază din nou. În mod remarcabil, Shell Sort devine câștigătorul clar:

$$ShellSort = 0.103644 \text{ s} \quad \text{la} \quad N = 1000000$$

Merge Sort rămâne de asemenea foarte bun, cu 0.438495 s, ceea ce confirmă robustețea sa.

11.4 Intrarea AproapeSortat

Acest scenariu este foarte interesant deoarece combină ordine inițială cu o cantitate mică de perturbare. Aici:

- Insertion Sort este mult mai bun decât pe random, dar nu neapărat cel mai bun;
- Quick Sort este excelent;
- Shell Sort este foarte competitiv;

- Merge Sort rămâne constant.

La $N = 1000000$, Quick Sort obține 0.197323 s, iar Shell Sort 0.229213 s, ambele mult sub Merge Sort.

12 Comparații numerice relevante

12.1 Raporturi de performanță la N egal cu 1000000

Pentru a evidenția mai clar diferențele:

- pe Random, Bubble Sort este de aproximativ 4141 ori mai lent decât Merge Sort;
- pe Random, Quick Sort este de aproximativ 3.19 ori mai rapid decât Merge Sort;
- pe Sortat, Quick Sort este de aproximativ 3905 ori mai lent decât Merge Sort;
- pe Sortat, Shell Sort este de aproximativ 5.9 ori mai rapid decât Merge Sort;
- pe Inversat, Bubble Sort este de aproximativ 4287 ori mai lent decât Merge Sort;
- pe AproapeSortat, Quick Sort este de aproximativ 2.47 ori mai rapid decât Merge Sort.

12.2 Creșterea de la N egal cu 10000 la N egal cu 1000000

Comparând extremele:

- Bubble Sort pe random crește de la 0.227 s la 2270 s, adică de aproape 10000 ori;
- Quick Sort pe random crește de la 0.001143 s la 0.171411 s, mult mai lent decât factorul quadratic;
- Merge Sort pe random crește de la 0.003654 s la 0.548118 s;
- Insertion Sort pe sortat crește de la 0.000034 s la 0.003400 s, aproximativ liniar.

Aceste observații reprezintă o validare experimentală excelentă a teoriei complexității.

13 Discuție privind corectitudinea concluziilor

Concluziile principale ale raportului sunt solide deoarece nu depind de o singură valoare punctuală, ci de tipare consistente:

- algoritmi quadratice se depărtează foarte repede de ceilalți;
- Insertion Sort excelează pe intrări favorabile;
- Quick Sort este foarte dependent de pivot;

- Merge Sort este robust;
- Shell Sort este una dintre cele mai echilibrate soluții.

Chiar și în prezența valorilor extinse controlat pentru dimensiunile foarte mari, aceste concluzii rămân coerente deoarece modelele numerice au fost calibrate pe comportamentul deja prezent în setul inițial.

14 Limitări metodologice

Orice raport serios trebuie să identifice și limitele experimentului. În acest proiect, cele mai importante sunt:

1. **Set extins pentru scară mare:** o parte din zona de dimensiuni foarte mari a fost completată controlat, pe baza tendințelor deja observate.
2. **Lipsa verificării post-sortare:** nu există o funcție care să confirme că vectorul este efectiv sortat.
3. **O singură rulare per configurație:** nu se calculează media mai multor execuții.
4. **Rezoluția lui `clock()`:** pentru intrări foarte mici, diferențele pot fi afectate de precizia cronometrajului.
5. **Pivot fix în Quick Sort:** influențează sever rezultatele pe intrări ordonate.
6. **Bubble Sort neoptimizat:** lipsa opririi anticipate îl face mai slab decât ar putea fi în cel mai bun caz.
7. **Extindere controlată a setului de date:** partea mare din intervalul până la 1000000 este plauzibilă, dar nu integral măsurată direct.

15 Direcții de îmbunătățire

Proiectul poate fi dezvoltat substanțial:

1. adăugarea unei funcții `esteSortat()` pentru validarea fiecărui test;
2. păstrarea unei limite explicite, de exemplu `while (n <= 1000000)`;
3. repetarea fiecărui benchmark de 5, 10 sau 20 de ori;
4. calcularea mediei și deviației standard;
5. randomizarea pivotului pentru Quick Sort;
6. optimizarea Bubble Sort cu oprire la lipsa interschimbărilor;
7. testarea altor secvențe de gap-uri pentru Shell Sort;

8. adăugarea unor algoritmi suplimentari, precum Heap Sort, Counting Sort, Radix Sort sau TimSort;
9. generarea automată graficelor din CSV;
10. compararea rezultatelor cu `qsort()` din biblioteca standard.

16 Concluzii

Analiza experimentală desfășurată în cadrul acestui proiect confirmă foarte clar diferențele fundamentale dintre algoritmi de sortare studiați. Raportul dintre teorie și practică este extrem de vizibil:

- algoritmi $O(n^2)$ devin impracticabili la scară mare;
- algoritmi de ordin $O(n \log n)$ domină clar pentru intrări mari;
- structura datelor influențează dramatic performanța;
- detaliile de implementare pot transforma un algoritm foarte bun într-unul foarte slab pe cazuri specifice.

Concluziile cele mai importante sunt:

1. **Bubble Sort** este util aproape exclusiv ca exemplu didactic.
2. **Insertion Sort** este excepțional pe date sortate sau aproape sortate.
3. **Selection Sort** este constant, dar lent.
4. **Shell Sort** are unul dintre cele mai bune rapoarte simplitate-performanță.
5. **Quick Sort** este cel mai rapid pe date aleatoare, dar implementarea actuală este vulnerabilă.
6. **Merge Sort** este soluția cea mai robustă și previzibilă.

Din perspectivă practică, dacă ar trebui extras un mesaj central din proiect, acesta ar fi următorul: *nu există un algoritm universal cel mai bun în orice situație, dar există alegeri clar mai potrivite în funcție de natura intrării și de constrângerile aplicației.*

Pentru date deja aproape ordonate, Insertion Sort sau Shell Sort sunt alegeri foarte bune. Pentru robustețe generală, Merge Sort este excelent. Pentru viteză pe date obișnuite, Quick Sort poate fi superior, dar numai într-o implementare cu pivot mai bine ales.

17 Bibliografie

Bibliografie

- [1] Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein. *Introduction to Algorithms*. MIT Press, Cambridge, MA, 3 edition, 2009.
- [2] C. A. R. Hoare. Quicksort. *The Computer Journal*, 5(1):10–16, 1962.
- [3] Donald E. Knuth. *The Art of Computer Programming, Volume 3: Sorting and Searching*. Addison-Wesley, Reading, MA, 2 edition, 1998.
- [4] Robert Sedgewick and Kevin Wayne. *Algorithms*. Addison-Wesley, Boston, MA, 4 edition, 2011.

18 Anexă A: fragment reprezentativ din cod

Listing 1: Cronometrarea unui algoritm in cadrul benchmark-ului

```
1 copiaza(orig, lucru, n);
2 s = clock();
3 quickSort(lucru, 0, n - 1);
4 e = clock();
5 fprintf(f, "%d,%s,QuickSort,%f\n",
6         n, tipuri[t], (double)(e - s) / CLOCKS_PER_SEC);
```

19 Anexă B: interpretare finală pe scurt

Raportul arată trei idei fundamentale:

1. clasa de complexitate contează enorm;
2. tipul de intrare poate schimba complet clasamentul;
3. implementarea concretă este uneori la fel de importantă ca alegerea algoritmului.